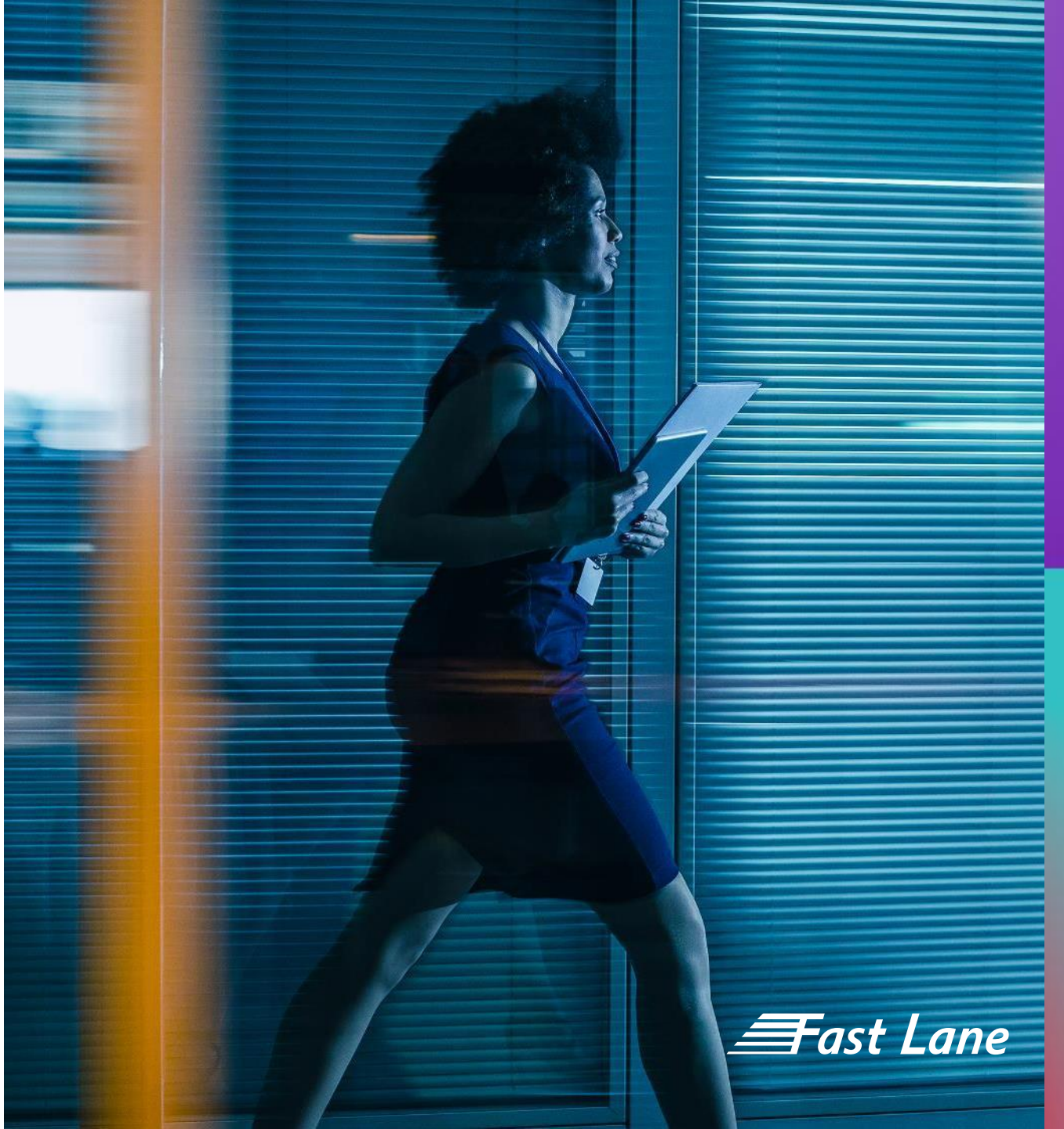
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 5

Events & Streams with Event Grid & Event Hubs

Connecting Contoso Logistics to partners and the live map

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Event Grid: Custom Topics & Subscriptions

Publish ParcelDelivered as a CloudEvent

02

Event Grid: Filters, Retries, Dead-Letter

Retry schedule, dead-letter storage, errors without retry

03

Event Grid Namespaces: Pull Delivery

Partners receive, acknowledge and release at their pace

04

Event Hubs: Partitions & Checkpoints

Keys, consumer groups, checkpoints and ownership

05

Event Hubs: Kafka-Compatible Streaming

Kafka clients on port 9093 with PLAIN or OAUTHBEARER

06

SignalR Service: Push to Browsers

Default and Serverless modes, the ops console client

Events and streams with Event Grid and Event Hubs

Connecting Contoso Logistics to partners and the live map

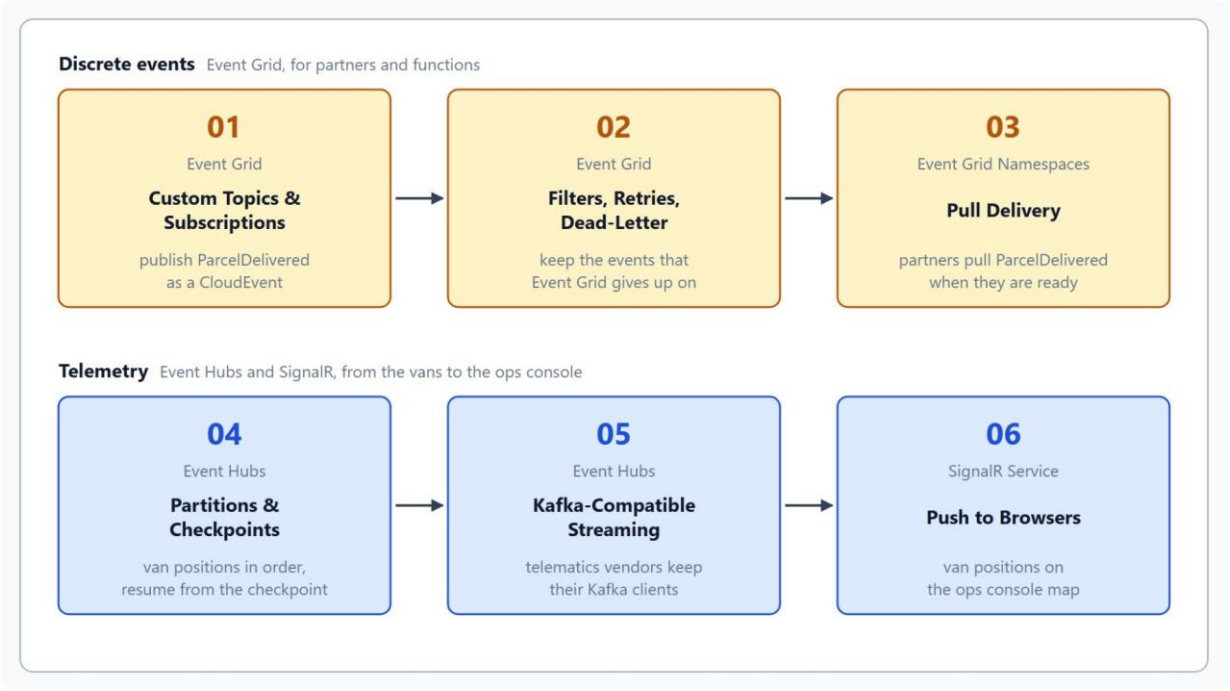
- **Discrete events**: publish `ParcelDelivered` with Event Grid, then filter, retry and pull it
- **Telemetry**: stream van positions through Event Hubs partitions, checkpoints and Kafka
- **Push to browsers**: SignalR Service brings each position to the ops console



Overview

The path from ParcelDelivered to the live map

- Publish `ParcelDelivered` to a **custom topic** and validate each partner endpoint
- Filter and retry deliveries, and **dead-letter** the ones that fail
- Let partners **pull** from a namespace topic when they are ready
- Stream van positions through **partitions**, checkpoints and the Kafka endpoint
- Push each position to the ops console with **SignalR Service**



01

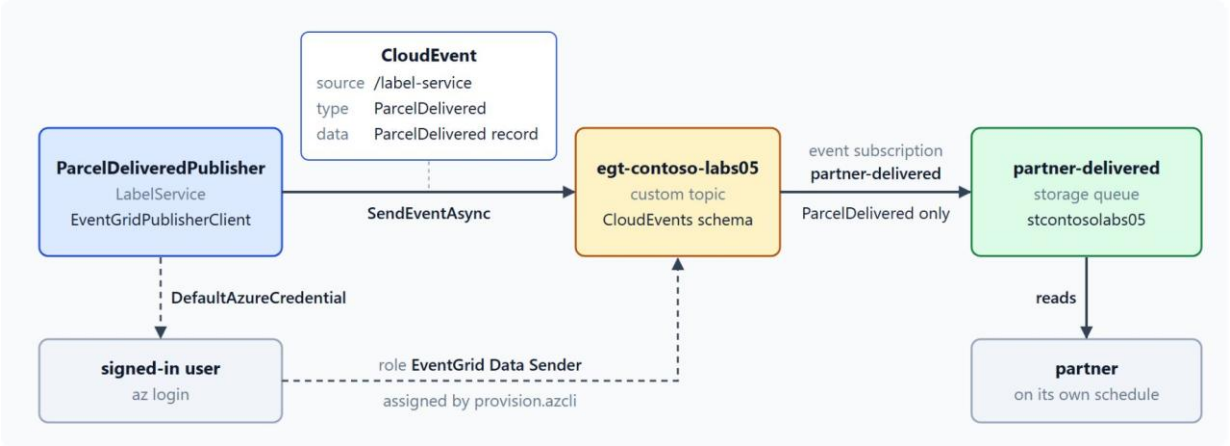
Event Grid: Custom Topics & Subscriptions

Publish ParcelDelivered as a CloudEvent

Publishing CloudEvents to a custom topic

The label-service publishes once, each partner gets a subscription

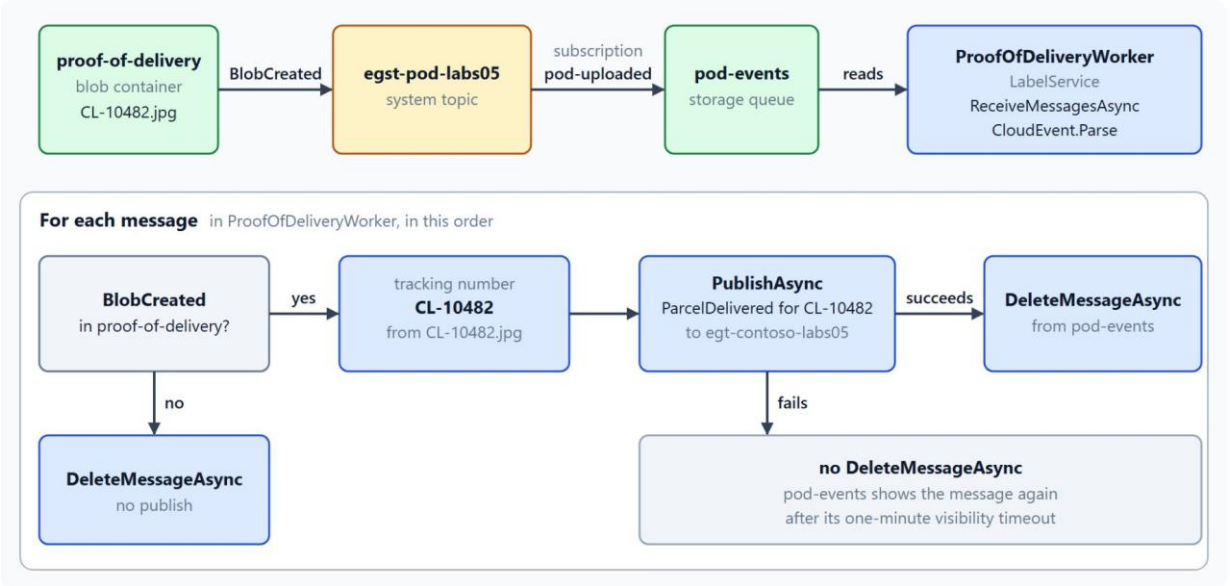
- A **custom topic** has an endpoint and keys; a system topic belongs to an Azure service
- `EventGridPublisherClient` sends a `CloudEvent` with a source, a type and data
- `DefaultAzureCredential` needs the **EventGrid Data Sender** role on the topic
- Handlers: HTTPS webhooks, Azure Functions, Service Bus, Event Hubs and Storage queues
- The event type name is the contract with every subscriber



Blob Storage system topic for proof of delivery

A finished photo upload becomes ParcelDelivered

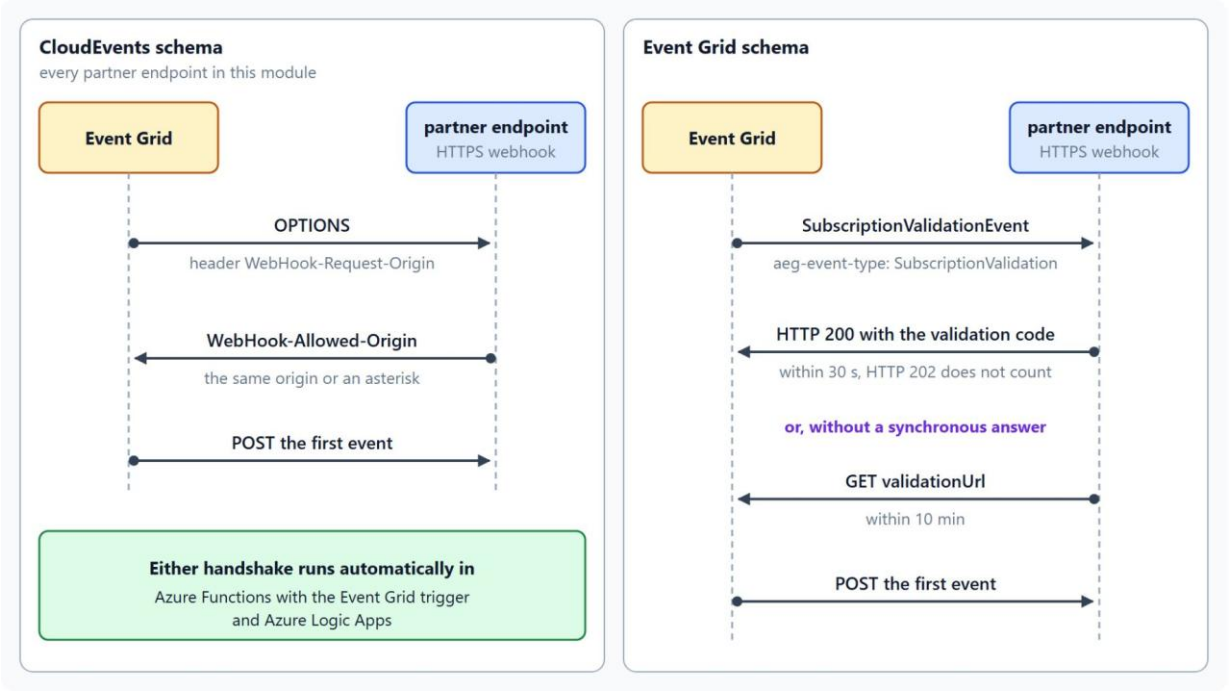
- Blob Storage raises `Microsoft.Storage.BlobCreated` once a block blob is fully committed
- A **system topic** turns those storage events into Event Grid events
- `subjectBeginsWith` limits the subscription to the proof-of-delivery container
- The worker reads the tracking number from the blob name and publishes `ParcelDelivered`
- Create the account as general purpose v2, because v1 can't publish to Event Grid



Endpoint validation

Event Grid proves you own the endpoint before the first event

- **CloudEvents schema**: `OPTIONS` with `WebHook-Request-Origin`, answered by `WebHook-Allowed-Origin`
- **Event Grid schema**: a `SubscriptionValidationEvent`, answered with HTTP 200 and the code within 30 seconds
- Without a synchronous answer, open the `validationUrl` with a GET within 10 minutes
- Azure Functions with the Event Grid trigger and Logic Apps answer automatically
- HTTP 202 doesn't count, and self-signed certificates aren't supported



02

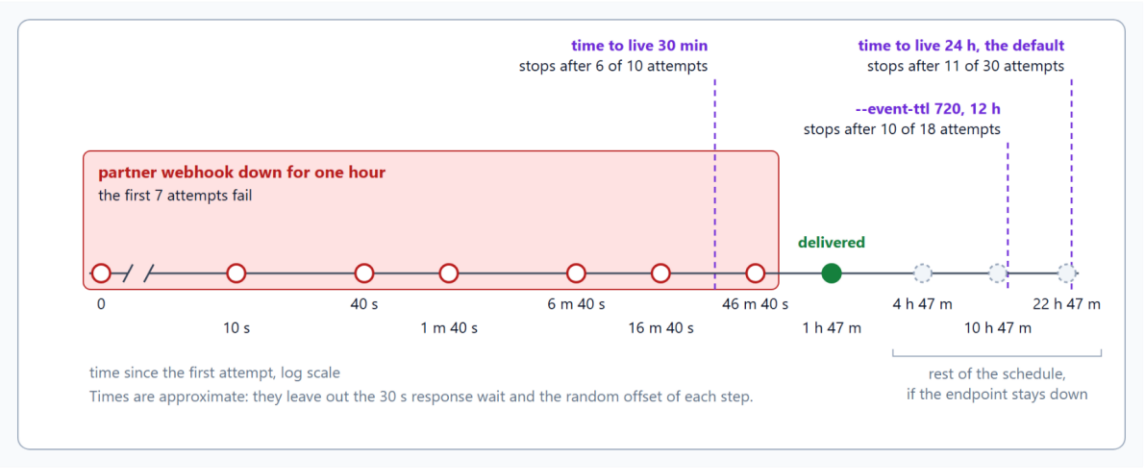
Event Grid: Filters, Retries, Dead-Letter

Retry schedule, dead-letter storage, errors without retry

Filters and the retry schedule

Which events reach a subscription, and how long Event Grid keeps trying

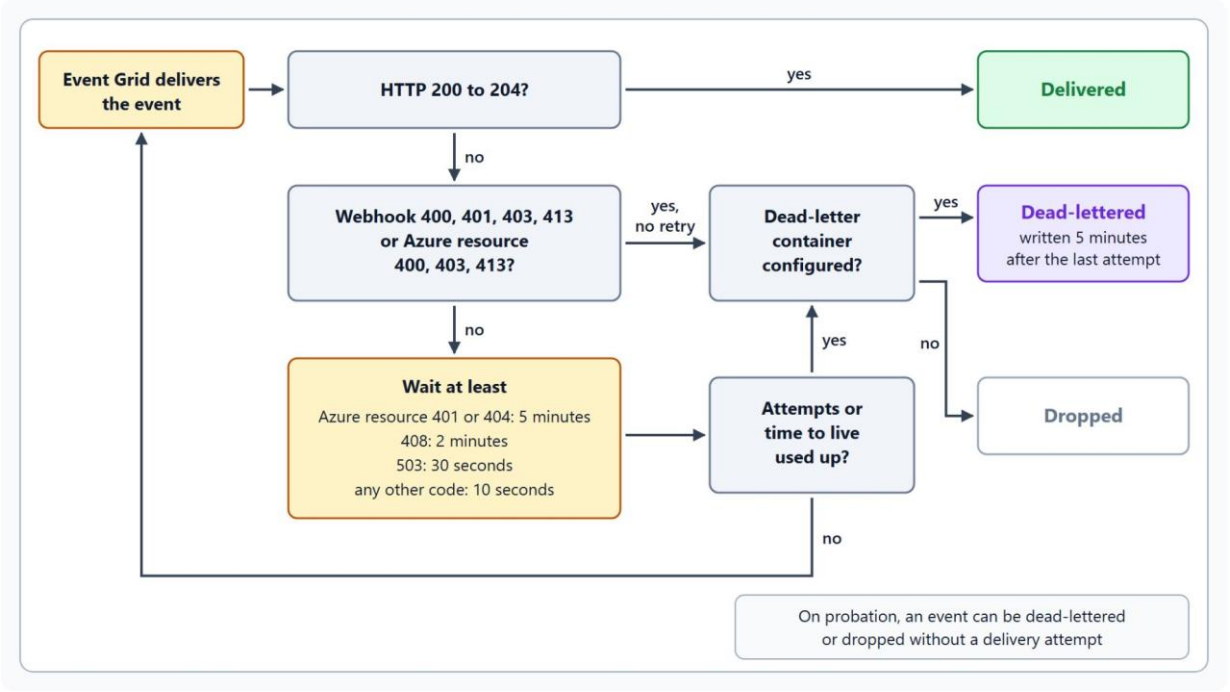
- Filter on **event type**, the start or end of the subject, and fields under `data`
- The **retry policy** allows 1 to 30 attempts and a time to live of 1 to 1440 minutes
- Event Grid stops at whichever limit it reaches first
- Waits grow from 10 seconds to every 12 hours, plus a small random offset
- Order isn't guaranteed and a retry can duplicate, so handlers accept both



Dead-lettering and errors without retries

Where an event ends up when delivery fails

- Dead-lettering is off by default: give the subscription a **blob container**
- Webhook 400, 401, 403 and 413 and Azure resource 400, 403 and 413 are never retried
- The dead-letter write comes five minutes after the last attempt
- Each event in the blob carries `deadLetterReason`, `deliveryAttempts` and `lastDeliveryOutcome`
- On **probation**, an event can be dead-lettered without a delivery attempt



03

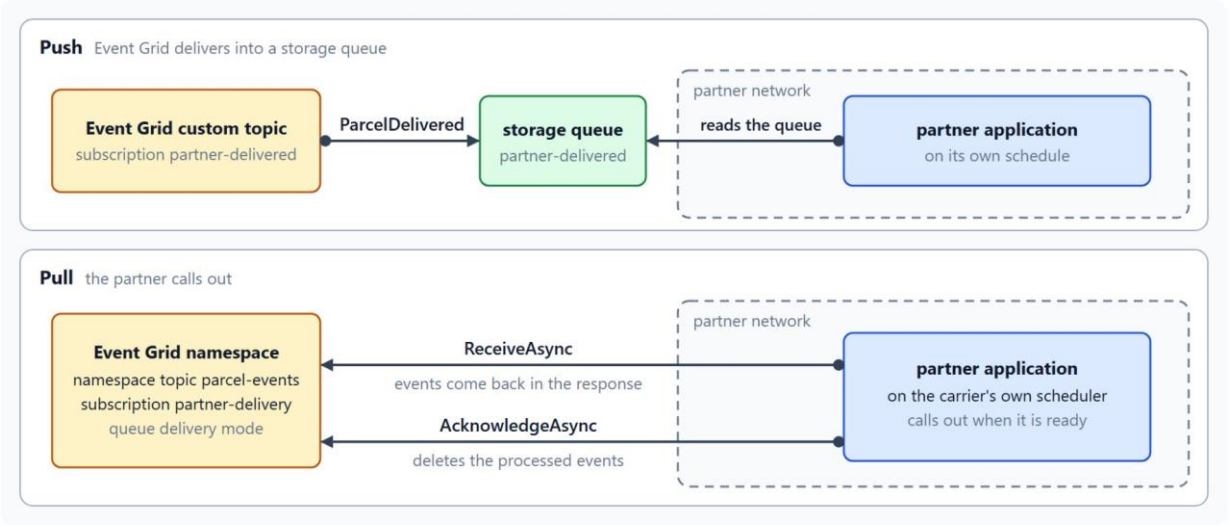
Event Grid Namespaces: Pull Delivery

Partners receive, acknowledge and release at their pace

Pull delivery with Event Grid namespaces

Partners without an inbound port call out when they are ready

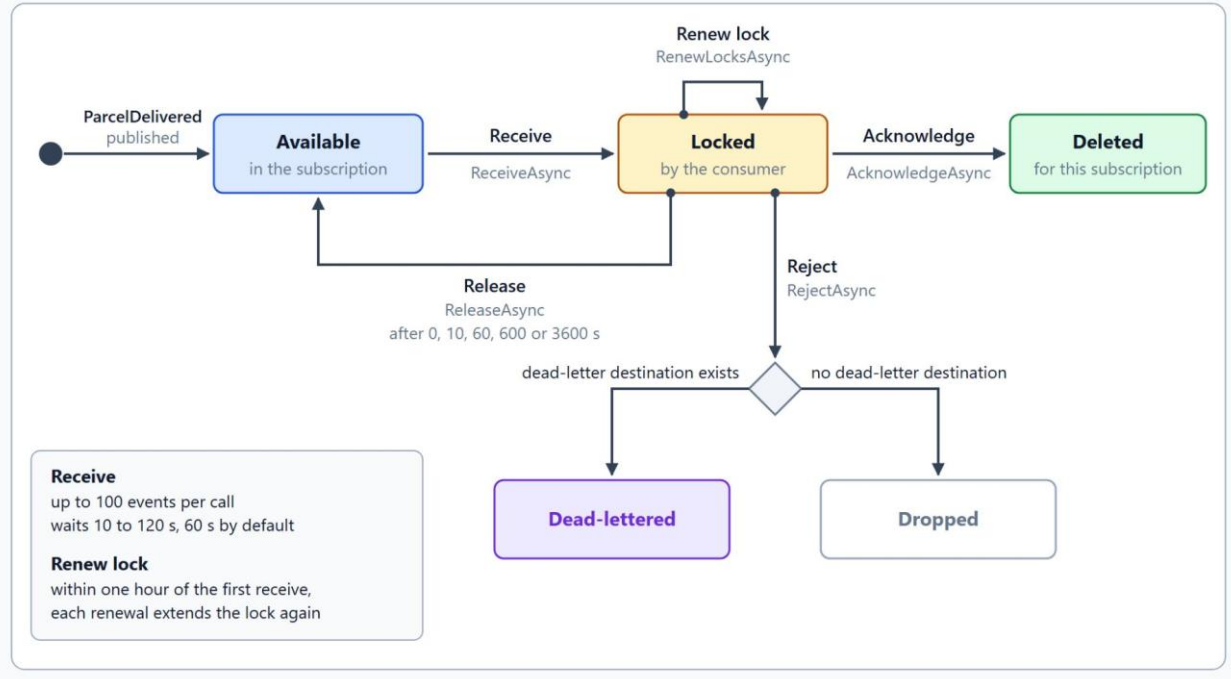
- A **namespace topic** receives CloudEvents 1.0; each subscription picks `queue` or `push`
- With pull, the partner calls out, so its network needs no inbound port
- Each partner gets its own subscription and its own state for every event
- Limits: 7 days retention, 500 subscriptions per topic, 1 MB per event
- Push mode delivers to Event Hubs, the only destination supported today



Lock tokens in pull delivery

Every received event stays locked until the consumer settles it

- `ReceiveAsync` reads up to 100 events and locks them for the consumer
- `AcknowledgeAsync` deletes the event for this subscription only
- `ReleaseAsync` makes it available again after 0, 10, 60, 600 or 3600 seconds
- `RejectAsync` dead-letters it, or drops it when no destination exists
- `RenewLocksAsync` extends the lock while the consumer still works on it



04

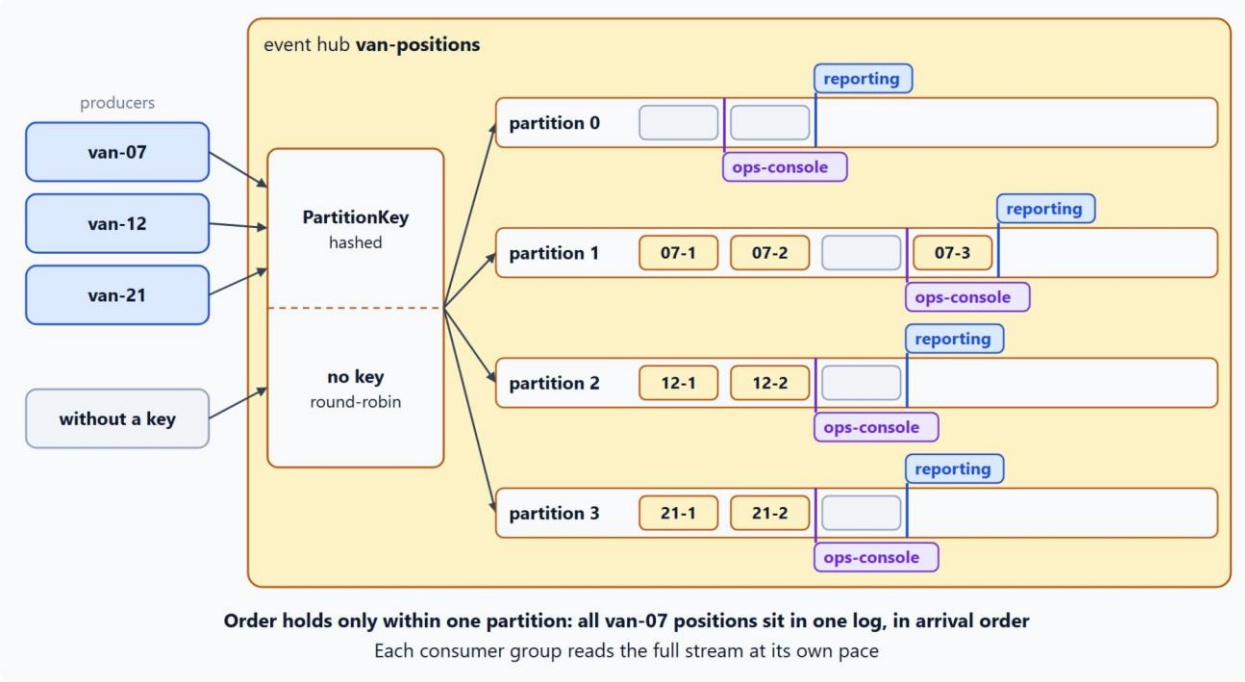
Event Hubs: Partitions & Checkpoints

Keys, consumer groups, checkpoints and ownership

Partitions and consumer groups

Order holds within one partition, each group reads the whole stream

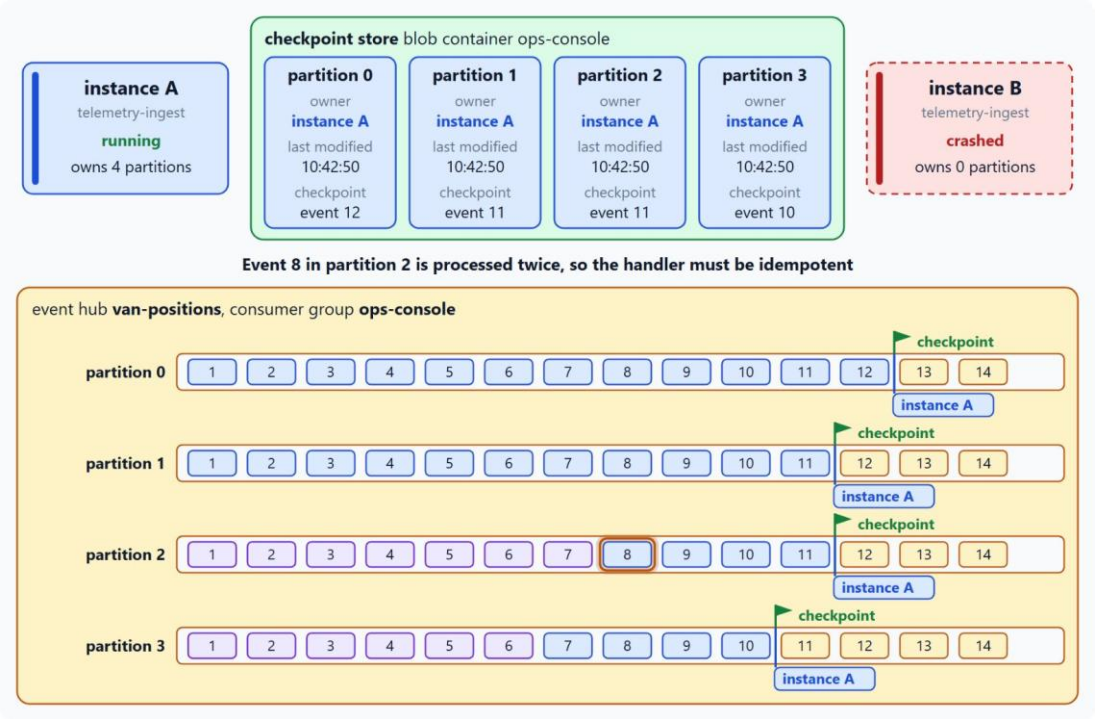
- Each **partition** is an ordered log, and order holds only inside one
- The van ID as partition key keeps one van's positions together, in order
- Events without a key go round-robin; a fixed partition ID lowers availability
- A **consumer group** reads the full stream at its own pace
- One active reader per partition and group, one checkpoint container per group



Checkpoints and partition ownership

A crashed instance's partitions resume from the last checkpoint

- `EventProcessorClient` stores ownership and checkpoints in a `BlobContainerClient`
- Instances claim partitions until the load is balanced
- After a crash, another instance takes over from the last checkpoint
- Delivery is **at least once**, so the handler must be idempotent
- **Capture** archives the stream to storage in Avro format



05

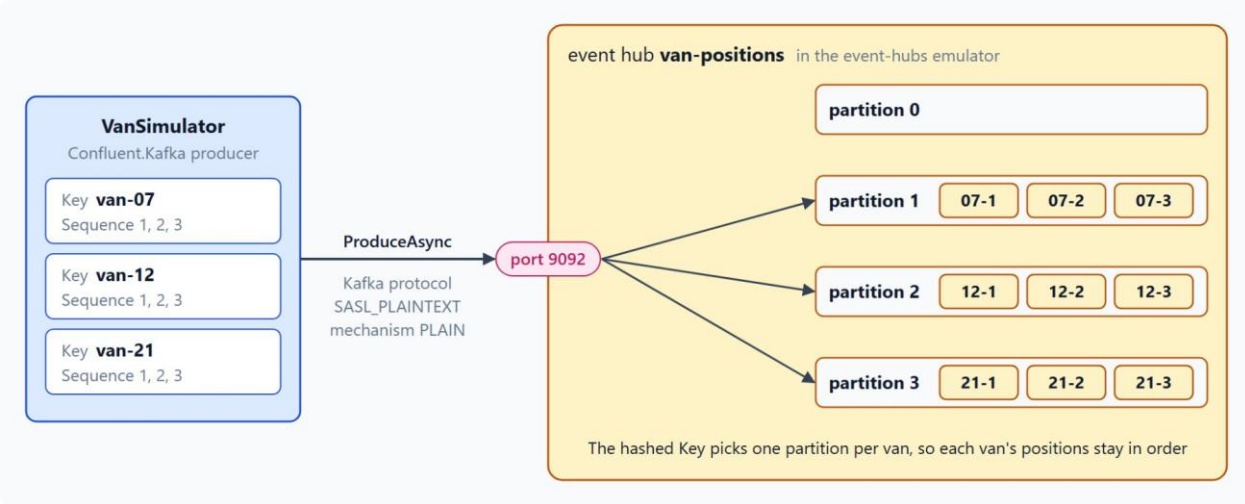
Event Hubs: Kafka-Compatible Streaming

Kafka clients on port 9093 with PLAIN or OAUTHBEARER

The Kafka endpoint of Event Hubs

Kafka clients keep their code and change their connection settings

- Port 9093 on the namespace (9092 in the emulator); a Kafka topic is an **event hub**
- Standard, Premium and Dedicated tiers; Basic has no Kafka endpoint
- `SASL_SSL` with **PLAIN** and a connection string, or **OAUTHBEARER** and an Entra ID token
- The message key picks the partition, so each van's positions stay in order
- Only gzip compression; treat Kafka Streams and transactions as preview



06

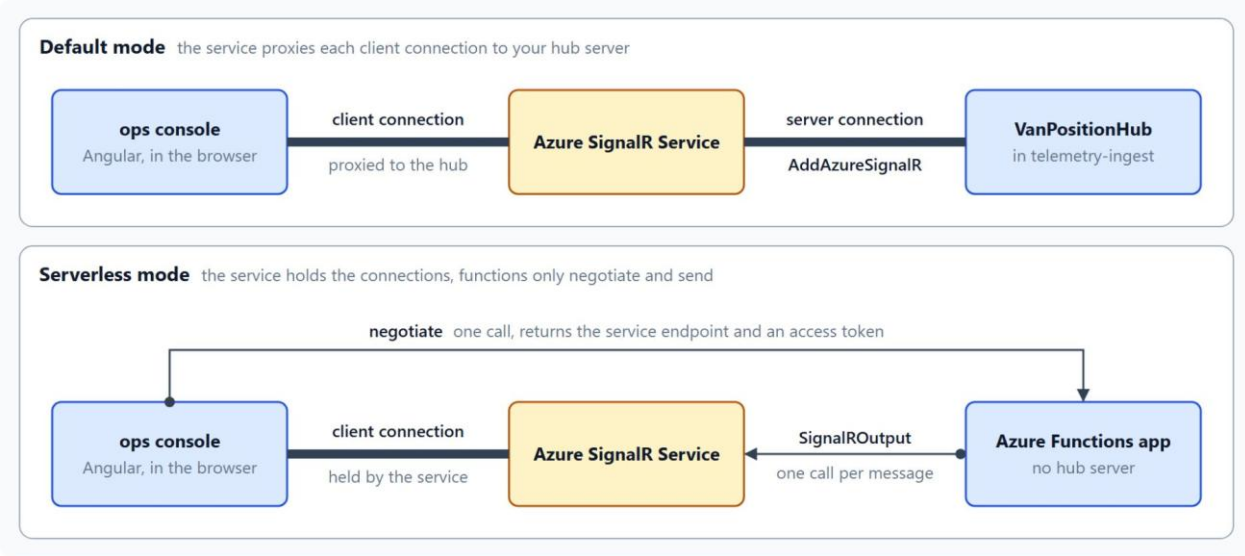
SignalR Service: Push to Browsers

Default and Serverless modes, the ops console client

SignalR Service modes

Who holds the browser connections

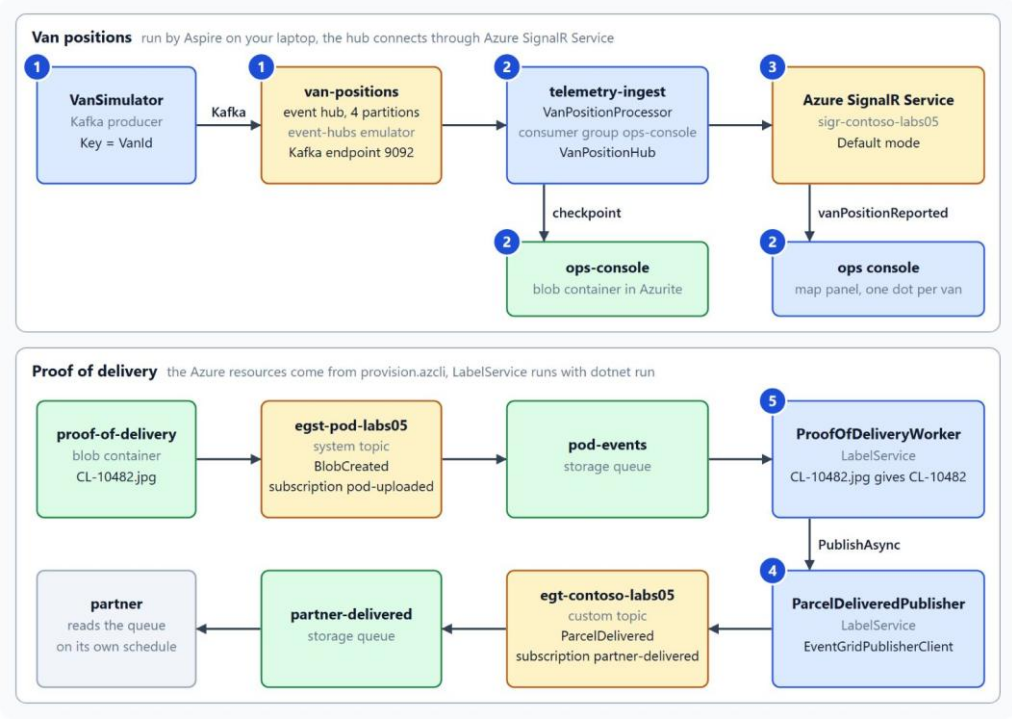
- **Default**: your server hosts the hub, and the service proxies each client to it
- `AddAzureSignalR` on the `AddSignalR` registration, only when the connection string is set
- **Serverless**: a `negotiate` function and `SignalROutput`, with no hub server
- `withAutomaticReconnect` retries after 0, 2, 10 and 30 seconds
- **Web PubSub** fits when you need a WebSocket protocol of your own



Streaming van positions to a live map

The lab in five steps, from the van simulator to the partner queue

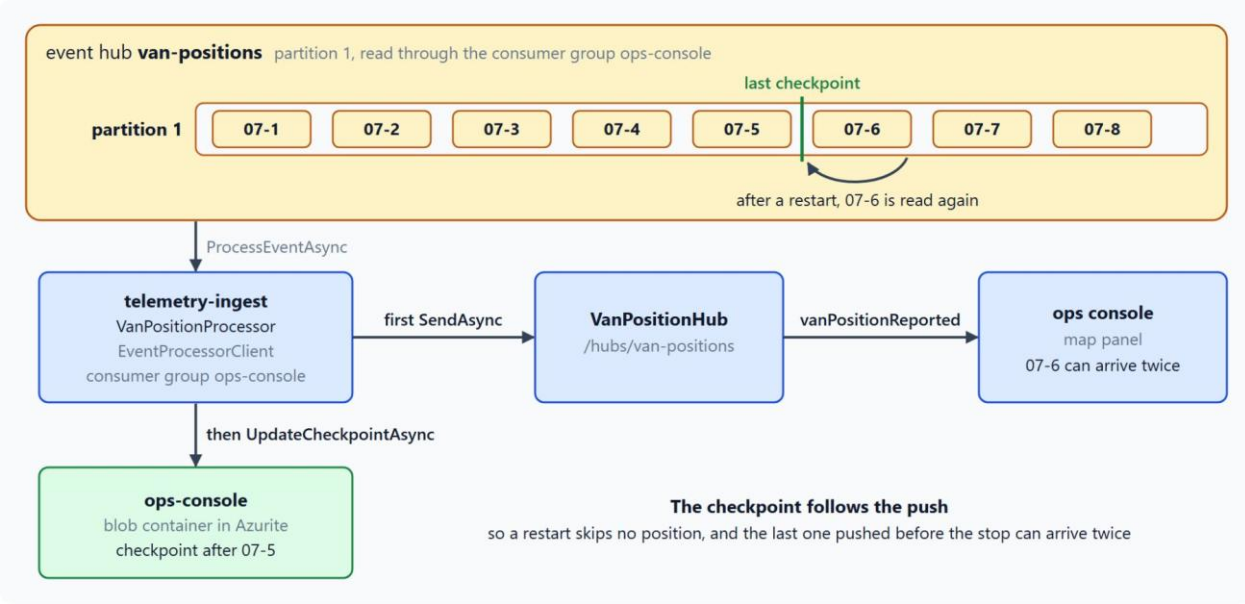
- The lab starts from the lab 4 solution, with an empty map and no partner events
- Step 1: a van simulator sends keyed positions over **Kafka**
- Steps 2 and 3: a checkpointed processor pushes them through **Azure SignalR Service**
- Steps 4 and 5: a proof-of-delivery photo publishes `ParcelDelivered` to the partner queue
- About 45 minutes, with Docker Desktop and `provision.azcli`



Checkpointing the van position processor

A restart replays from the last checkpoint and skips no position

- `VanPositionProcessor` reads `van-positions` through the `ops-console` consumer group
- Each position goes out with `SendAsync` before `UpdateCheckpointAsync` stores the checkpoint
- Checkpoints live in the `ops-console` blob container in Azurite
- After a restart, the last pushed position can reach the map twice
- Test it: stop and start `telemetry-ingest`, and every van's sequence continues without a gap



Takeaways

What to carry into event sourcing and CQRS

- Publish `ParcelDelivered` once and keep its event type stable
- Give every subscription filters, a retry policy and a **dead-letter** container
- Offer **pull** delivery to partners that can't open an inbound port
- Key by van ID, read with one consumer group per service, keep handlers idempotent
- Push to browsers through **SignalR Service** in Default mode

